

CSS clip-path Property: Complete Guide

Master Shape Clipping and Visual Effects

Contents

1. Introduction	
(a) What is clip-path?	2
(b) clip-path vs. mask	2
2. Syntax and Basics	3
(a) Function Syntax	3
i. Basic Structure	3
ii. Examples	3
(b) Property Values	4
i. Available Functions	4
ii. Examples	4
3. Basic Shapes	5
(a) inset() - Rectangular Clipping	5
i. Syntax and Usage	5
ii. Examples	5
(b) circle() - Circular Clipping	6
i. Radius and Position	6
ii. Examples	7
(c) ellipse() - Elliptical Clipping	7
i. Radii and Center	7
ii. Examples	8
(d) polygon() - Custom Shapes	8
i. Coordinate Definition	8
ii. Examples	9
4. Advanced Clipping	9
(a) path() - SVG Paths	9
i. SVG Syntax	9
ii. Examples	10
(b) margin-box and border-box	10

i. Reference Boxes	10
ii. Examples	11
5. Real-World Use Cases	11
(a) Use Case 1: Image Effects	11
(b) Use Case 2: Hero Section Designs	12
(c) Use Case 3: Animated Reveals	12
(d) Use Case 4: Creative UI Elements	13
6. Animations with clip-path	14
(a) Animating Shapes	14
i. Keyframe Animations	14
ii. Examples	14
(b) Hover Effects	15
i. Interactive Clipping	15
ii. Examples	15
(c) Transition Timing	16
i. Smooth Transitions	16
ii. Examples	16
7. Performance and Optimization	16
(a) Browser Considerations	16
(b) Best Practices	17
8. Browser Support & Compatibility	17
(a) Feature Support	17
(b) Vendor Prefixes	18
9. Common Mistakes & Solutions	18
(a) Mistake 1: Incorrect Coordinate System	18
(b) Mistake 2: Over-Clipping Content	19
(c) Mistake 3: Animation Incompatibilities	19
10. Summary: Key Takeaways	19
11. Conclusion	20

1 Introduction

The CSS `clip-path` property clips portions of an element using a defined shape. It allows you to create complex visual effects, custom shapes, and reveals without using image editing tools or additional markup.

1.1 What is clip-path?

`clip-path` defines a clipping region that determines which parts of an element are visible and which are hidden.

Key characteristics:

- Clips element content using geometric shapes
- Supports rectangles, circles, ellipses, polygons, and SVG paths
- Hides clipped areas without removing content from DOM
- Works with all HTML elements
- Fully animatable for dynamic effects
- No additional markup or elements required
- GPU-accelerated for smooth performance

1.2 clip-path vs. mask

Understanding the difference between `clip-path` and `mask`:

Aspect	clip-path	mask
Clipping	Sharp edges	Soft transitions
Grayscale	No	Yes (opacity levels)
Performance	Faster	Slower
Use case	Shapes	Images/gradients
Browser support	Wider	Limited

2 Syntax and Basics

2.1 Function Syntax

`clip-path` uses a straightforward function-based syntax.

2.1.1 Basic Structure

```
/* Basic syntax */
clip-path: shape(parameters);

/* Examples */
clip-path: circle(50%);
clip-path: polygon(0 0, 100% 0, 100% 100%, 0 100%);
clip-path: inset(10px 20px 30px 40px);
clip-path: ellipse(40% 50%);
```

Key rules:

- Use percentage or absolute units
- Coordinates are relative to element dimensions
- 0,0 is top-left corner
- 100
- Percentages refer to element width/height

2.1.2 Examples

Example 1: Basic Clip-Path

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .circle-clip {
      width: 200px;
      height: 200px;
      background: #007BFF;
      clip-path: circle(50%);
    }

    .polygon-clip {
      width: 200px;
      height: 200px;
      background: #FF6B6B;
      clip-path: polygon(50% 0%, 100% 50%, 50% 100%, 0% 50%);
    }
  </style>
</head>
<body>
  <div class="circle-clip"></div>
  <div class="polygon-clip"></div>
</body>
</html>
```

2.2 Property Values

Available clip-path functions and values.

2.2.1 Available Functions

```
/* Built-in shapes */
clip-path: inset(...);           /* Rectangle with optional rounded corners */
clip-path: circle(...);         /* Perfect circle */
clip-path: ellipse(...);        /* Oval shape */
clip-path: polygon(...);        /* Custom polygon */
clip-path: path(...);           /* SVG path */

/* Special values */
clip-path: auto;                 /* No clipping (default) */
clip-path: none;                 /* No clipping */

/* Box model reference */
clip-path: border-box;
clip-path: padding-box;
clip-path: content-box;
clip-path: margin-box;
```

2.2.2 Examples

Example 1: Different Shapes

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .shape {
      width: 150px;
      height: 150px;
      margin: 20px;
      background: #007BFF;
    }

    .inset { clip-path: inset(20px); }
    .circle { clip-path: circle(50%); }
    .ellipse { clip-path: ellipse(50% 30%); }
    .polygon { clip-path: polygon(50% 0%, 100% 38%, 82% 100%, 18% 100%, 0%
      38%); }
  </style>
</head>
<body>
  <div class="shape inset">Inset</div>
  <div class="shape circle">Circle</div>
  <div class="shape ellipse">Ellipse</div>
  <div class="shape polygon">Star</div>
</body>
</html>
```

3 Basic Shapes

3.1 inset() - Rectangular Clipping

The `inset()` function clips away outer edges of an element.

3.1.1 Syntax and Usage

```
/* inset(top right bottom left) - like margin/padding */
clip-path: inset(10px);                /* All sides */
clip-path: inset(10px 20px);           /* Vertical Horizontal */
clip-path: inset(10px 20px 30px 40px); /* Top Right Bottom Left */

/* With rounded corners */
clip-path: inset(10px round 5px);
```

3.1.2 Examples

Example 1: Simple Inset

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .inset-basic {
      width: 300px;
      height: 200px;
      background: url('image.jpg');
      background-size: cover;
      clip-path: inset(20px);
    }

    .inset-asymmetric {
      width: 300px;
      height: 200px;
      background: #007BFF;
      clip-path: inset(10px 40px 30px 20px);
    }
  </style>
</head>
<body>
  <div class="inset-basic">Symmetric Inset</div>
  <div class="inset-asymmetric">Asymmetric Inset</div>
</body>
</html>
```

Example 2: Inset with Rounded Corners

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .inset-rounded {
```

```
        width: 300px;
        height: 200px;
        background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
        clip-path: inset(15px round 20px);
    }
</style>
</head>
<body>
    <div class="inset-rounded">Rounded Inset</div>
</body>
</html>
```

3.2 circle() - Circular Clipping

Create perfect circular clipping regions.

3.2.1 Radius and Position

```
/* circle(radius) - centered by default */
clip-path: circle(50%);
clip-path: circle(100px);
clip-path: circle(50% at 50% 50%);          /* Explicit center */

/* Positioned circles */
clip-path: circle(30% at 25% 25%);          /* Top-left positioned */
clip-path: circle(40% at 80% 20%);          /* Top-right positioned */
```

3.2.2 Examples

Example 1: Centered Circle

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .circle-image {
            width: 200px;
            height: 200px;
            background: url('image.jpg');
            background-size: cover;
            clip-path: circle(50%);
        }
    </style>
</head>
<body>
    <div class="circle-image"></div>
</body>
</html>
```

Example 2: Positioned Circles

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .circle-top-left {
      width: 200px;
      height: 200px;
      background: #FF6B6B;
      clip-path: circle(40% at 30% 30%);
    }

    .circle-bottom-right {
      width: 200px;
      height: 200px;
      background: #4ECDC4;
      clip-path: circle(40% at 70% 70%);
    }
  </style>
</head>
<body>
  <div class="circle-top-left">Top-Left</div>
  <div class="circle-bottom-right">Bottom-Right</div>
</body>
</html>
```

3.3 ellipse() - Elliptical Clipping

Create oval-shaped clipping regions.

3.3.1 Radii and Center

```
/* ellipse(rx ry) - centered by default */
clip-path: ellipse(50% 30%);
clip-path: ellipse(100px 60px);
clip-path: ellipse(50% 30% at 50% 50%); /* Explicit center */

/* Positioned ellipses */
clip-path: ellipse(40% 50% at 30% 30%); /* Top-left */
clip-path: ellipse(60% 40% at 70% 60%); /* Bottom-right */
```

3.3.2 Examples

Example 1: Basic Ellipse

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .ellipse-wide {
      width: 300px;
```



```

        height: 200px;
        background: #007BFF;
        clip-path: ellipse(60% 40%);
    }

    .ellipse-tall {
        width: 200px;
        height: 300px;
        background: #FF6B6B;
        clip-path: ellipse(40% 60%);
    }
</style>
</head>
<body>
    <div class="ellipse-wide">Wide Ellipse</div>
    <div class="ellipse-tall">Tall Ellipse</div>
</body>
</html>

```

Example 2: Positioned Ellipse

```

<!DOCTYPE html>
<html>
<head>
    <style>
        .ellipse-positioned {
            width: 300px;
            height: 300px;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            clip-path: ellipse(50% 30% at 50% 40%);
        }
    </style>
</head>
<body>
    <div class="ellipse-positioned">Positioned Ellipse</div>
</body>
</html>

```

3.4 polygon() - Custom Shapes

Create custom multi-sided shapes using coordinates.

3.4.1 Coordinate Definition

```

/* polygon(x1 y1, x2 y2, x3 y3, ...) */
clip-path: polygon(50% 0%, 100% 50%, 50% 100%, 0% 50%); /* Diamond */
clip-path: polygon(0% 0%, 100% 0%, 100% 100%, 0% 100%); /* Square */
clip-path: polygon(50% 0%, 100% 38%, 82% 100%, 18% 100%, 0% 38%); /* Star */

/* Triangle */
clip-path: polygon(50% 0%, 100% 100%, 0% 100%);

```

```
/* Hexagon */
clip-path: polygon(50% 0%, 100% 25%, 100% 75%, 50% 100%, 0% 75%, 0% 25%);
```

3.4.2 Examples

Example 1: Common Shapes

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .shape {
      width: 150px;
      height: 150px;
      margin: 20px;
      background: #007BFF;
    }

    .triangle {
      clip-path: polygon(50% 0%, 100% 100%, 0% 100%);
    }

    .diamond {
      clip-path: polygon(50% 0%, 100% 50%, 50% 100%, 0% 50%);
    }

    .hexagon {
      clip-path: polygon(50% 0%, 100% 25%, 100% 75%, 50% 100%, 0% 75%,
        0% 25%);
    }
  </style>
</head>
<body>
  <div class="shape triangle">Triangle</div>
  <div class="shape diamond">Diamond</div>
  <div class="shape hexagon">Hexagon</div>
</body>
</html>
```

Example 2: Star Shape

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .star {
      width: 200px;
      height: 200px;
      background: #FFD700;
      clip-path: polygon(
        50% 0%, 61% 35%, 98% 35%, 68% 57%, 79% 91%,
        50% 70%, 21% 91%, 32% 57%, 2% 35%, 39% 35%
      );
    }
  </style>
</head>
<body>
  <div class="star">Star</div>
</body>
</html>
```

```
        );  
    }  
    </style>  
</head>  
<body>  
    <div class="star">Star</div>  
</body>  
</html>
```

4 Advanced Clipping

4.1 path() - SVG Paths

Use SVG path data for precise custom clipping.

4.1.1 SVG Syntax

```
/* SVG path syntax */
clip-path: path('M 0,0 L 100,0 L 100,100 L 0,100 Z');

/* Path with curves */
clip-path: path('M 10,80 C 40,10, 65,10, 95,80 S 150,150, 180,80');

/* Complex paths */
clip-path: path('M 50,0 Q 0,0, 0,50 Q 0,100, 50,100 Q 100,100, 100,50 Q 100,0,
50,0');
```

4.1.2 Examples

Example 1: Basic SVG Path

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .wave-clip {
      width: 300px;
      height: 200px;
      background: #007BFF;
      clip-path: path(
        'M 0,100 Q 75,50, 150,100 T 300,100 L 300,200 L 0,200 Z'
      );
    }
  </style>
</head>
<body>
  <div class="wave-clip">Wave Shape</div>
</body>
</html>
```

4.2 margin-box and border-box

Control which box model area to clip.

4.2.1 Reference Boxes

```
/* Box model options */
clip-path: inset(10px) border-box;      /* Clips relative to border */
clip-path: circle(50%) padding-box;     /* Clips relative to padding */
clip-path: polygon(...) content-box;    /* Clips relative to content */
```

```
clip-path: circle(50%) margin-box;      /* Clips relative to margin */
```

4.2.2 Examples

Example 1: Different Box Models

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .box {
      width: 200px;
      height: 200px;
      margin: 20px;
      padding: 20px;
      border: 5px solid #333;
      background: #007BFF;
    }

    .border-box {
      clip-path: circle(50%) border-box;
    }

    .padding-box {
      clip-path: circle(50%) padding-box;
    }

    .content-box {
      clip-path: circle(50%) content-box;
    }
  </style>
</head>
<body>
  <div class="box border-box">Border Box</div>
  <div class="box padding-box">Padding Box</div>
  <div class="box content-box">Content Box</div>
</body>
</html>
```

5 Real-World Use Cases

5.1 Use Case 1: Image Effects

Create stylish image treatments with clip-path.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .image-container {
      width: 300px;
```

```
        height: 300px;
        overflow: hidden;
    }

    .hexagon-image {
        width: 100%;
        height: 100%;
        background: url('portrait.jpg');
        background-size: cover;
        clip-path: polygon(50% 0%, 100% 25%, 100% 75%, 50% 100%, 0% 75%,
            0% 25%);
    }

    .circle-image {
        width: 100%;
        height: 100%;
        background: url('avatar.jpg');
        background-size: cover;
        clip-path: circle(50%);
    }
}
</style>
</head>
<body>
    <div class="image-container">
        <div class="hexagon-image"></div>
    </div>
    <div class="image-container">
        <div class="circle-image"></div>
    </div>
</body>
</html>
```

5.2 Use Case 2: Hero Section Designs

Create eye-catching hero sections.

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .hero {
            width: 100%;
            height: 500px;
            background: url('hero.jpg') center/cover;
            clip-path: polygon(0 0, 100% 0, 100% 85%, 0 100%);
            display: flex;
            align-items: center;
            justify-content: center;
            color: white;
            font-size: 2rem;
        }
    </style>
</head>
<body>
```



```
        color: white;
        font-weight: bold;
    }
</style>
</head>
<body>
    <div class="ribbon">Ribbon</div>
</body>
</html>
```


6 Animations with clip-path

6.1 Animating Shapes

clip-path is fully animatable using CSS animations.

6.1.1 Keyframe Animations

```
@keyframes morph {
  0% {
    clip-path: circle(50%);
  }
  50% {
    clip-path: ellipse(40% 60%);
  }
  100% {
    clip-path: polygon(50% 0%, 100% 38%, 82% 100%, 18% 100%, 0% 38%);
  }
}

.morphing {
  animation: morph 3s infinite;
}
```

6.1.2 Examples

Example 1: Shape Morphing

```
<!DOCTYPE html>
<html>
<head>
  <style>
    @keyframes shape-morph {
      0% { clip-path: circle(50%); }
      25% { clip-path: ellipse(60% 40%); }
      50% { clip-path: polygon(50% 0%, 100% 50%, 50% 100%, 0% 50%); }
      75% { clip-path: polygon(50% 0%, 100% 38%, 82% 100%, 18% 100%, 0%
        38%); }
      100% { clip-path: circle(50%); }
    }

    .morph-shape {
      width: 200px;
      height: 200px;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      animation: shape-morph 4s infinite;
    }
  </style>
</head>
<body>
  <div class="morph-shape">Morphing Shape</div>
</body>
</html>
```

6.2 Hover Effects

Create interactive clip-path effects on hover.

6.2.1 Interactive Clipping

```
.hover-clip {  
    clip-path: circle(30%);  
    transition: clip-path 0.3s ease;  
}  
  
.hover-clip:hover {  
    clip-path: circle(50%);  
}
```

6.2.2 Examples

Example 1: Expanding Circle on Hover

```
<!DOCTYPE html>  
<html>  
<head>  
    <style>  
        .image-hover {  
            width: 250px;  
            height: 250px;  
            background: url('image.jpg') center/cover;  
            clip-path: circle(30%);  
            transition: clip-path 0.4s cubic-bezier(0.25, 0.46, 0.45, 0.94);  
            cursor: pointer;  
        }  
  
        .image-hover:hover {  
            clip-path: circle(50%);  
        }  
    </style>  
</head>  
<body>  
    <div class="image-hover"></div>  
</body>  
</html>
```

Example 2: Polygon Shape Reveal

```
<!DOCTYPE html>  
<html>  
<head>  
    <style>  
        .polygon-reveal {  
            width: 300px;  
            height: 300px;  
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);  
            clip-path: polygon(50% 50%, 50% 50%, 50% 50%, 50% 50%, 50% 50%);  
        }  
    </style>  
</head>  
<body>  
    <div class="polygon-reveal"></div>  
</body>  
</html>
```

```
        transition: clip-path 0.5s ease;
    }

    .polygon-reveal:hover {
        clip-path: polygon(0 0, 100% 0, 100% 100%, 0 100%);
    }
</style>
</head>
<body>
    <div class="polygon-reveal">Polygon Reveal</div>
</body>
</html>
```

6.3 Transition Timing

Control animation speed and timing.

6.3.1 Smooth Transitions

```
.clip-animation {
    clip-path: inset(0%);
    transition: clip-path 0.3s ease-out;
}

.clip-animation.active {
    clip-path: inset(10%);
    transition: clip-path 0.6s cubic-bezier(0.68, -0.55, 0.265, 1.55);
}
```

6.3.2 Examples

Example 1: Timing Functions

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .clip-ease {
            width: 200px;
            height: 200px;
            background: #007BFF;
            clip-path: circle(30%);
            transition: clip-path 0.5s ease;
        }

        .clip-ease:hover {
            clip-path: circle(50%);
        }

        .clip-linear {
            clip-path: circle(30%);
        }
    </style>
</head>
</html>
```

```
        transition: clip-path 0.5s linear;
    }

    .clip-linear:hover {
        clip-path: circle(50%);
    }

    .clip-cubic {
        clip-path: circle(30%);
        transition: clip-path 0.5s cubic-bezier(0.68, -0.55, 0.265, 1.55);
    }

    .clip-cubic:hover {
        clip-path: circle(50%);
    }
</style>
</head>
<body>
    <div class="clip-ease">Ease</div>
    <div class="clip-linear">Linear</div>
    <div class="clip-cubic">Cubic</div>
</body>
</html>
```

7 Performance and Optimization

7.1 Browser Considerations

clip-path performance varies across browsers.

- Supported in all modern browsers
- GPU-accelerated in most browsers
- SVG paths may have lower performance
- Complex polygons can impact performance
- Animations are generally smooth at 60fps
- Mobile performance acceptable for simple shapes

7.2 Best Practices

Optimize clip-path usage:

- Use simple shapes when possible
- Prefer circles/ellipses over complex polygons
- Avoid animating too many clipped elements simultaneously

- Use GPU-accelerated animations
- Test performance on target devices
- Consider fallback for older browsers
- Use `will-change` for animated elements

Performance optimization example:

```
.will-animate {  
  will-change: clip-path;  
  clip-path: circle(50%);  
}  
  
.will-animate:hover {  
  clip-path: circle(30%);  
}
```

8 Browser Support & Compatibility

8.1 Feature Support

Feature	Chrome	Firefox	Safari	Edge	Opera
Basic shapes	Yes	Yes	Yes	Yes	Yes
SVG path()	Yes	Partial	Limited	Yes	Yes
Animations	Yes	Yes	Yes	Yes	Yes

All modern browsers support basic clip-path shapes. SVG paths have more limited support.

8.2 Vendor Prefixes

The `-webkit-` prefix may be needed for older browser versions.

```
-webkit-clip-path: circle(50%);  
clip-path: circle(50%);
```

9 Common Mistakes & Solutions

9.1 Mistake 1: Incorrect Coordinate System

Problem: Misunderstanding coordinate positioning.

```
/* Wrong: Thinking coordinates as pixels */  
clip-path: polygon(50px 50px, 150px 50px, 150px 150px);  
  
/* Correct: Using percentages relative to element size */  
clip-path: polygon(33% 33%, 66% 33%, 66% 66%);
```

9.2 Mistake 2: Over-Clipping Content

Problem: Clipping away too much content unintentionally.

```
/* Wrong: Circle too small */
.image {
  width: 300px;
  height: 300px;
  background: url('image.jpg') center/cover;
  clip-path: circle(10%); /* Almost nothing visible */
}

/* Correct: Appropriate clipping */
.image {
  clip-path: circle(50%); /* Full visibility with shape */
}
```

9.3 Mistake 3: Animation Incompatibilities

Problem: Trying to animate between incompatible shapes.

```
/* Problematic: Different shape types won't morph smoothly */
@keyframes bad-morph {
  0% { clip-path: circle(50%); }
  100% { clip-path: inset(20%); } /* Abrupt change */
}

/* Better: Morph between compatible shapes */
@keyframes good-morph {
  0% { clip-path: circle(30%); }
  100% { clip-path: circle(50%); } /* Smooth transition */
}
```

10 Summary: Key Takeaways

1. **Understand coordinate system:** Percentages relative to element dimensions
2. **Choose appropriate shape:** circle, ellipse, polygon, inset, or path
3. **Use simple shapes for performance:** Circles faster than complex polygons
4. **Animate smoothly:** Transition between similar shape types
5. **Test cross-browser:** Support varies for advanced features
6. **Combine with transitions:** Create interactive hover effects
7. **Reference box models:** Choose appropriate reference area
8. **Optimize animations:** Use will-change for GPU acceleration

11 Conclusion

The CSS `clip-path` property opens up creative possibilities for web design. From simple circular images to complex animated shapes, `clip-path` provides the tools to create visually striking interfaces without requiring images or additional markup.

By mastering the different shape functions and animation techniques, you can elevate your design work and create memorable user experiences. Whether building hero sections, image galleries, or interactive UI elements, `clip-path` is an invaluable tool in modern CSS development!